Game Concept and Design Document

HIJIN-BATTLE

Student name: Xiangtian Ren

Student ID: 2617324

Unity version: 2022.3.62f3c1

Repository: https://github.com/Xiangtian-Gmae-program/Game_project

Repository folder: FINAL VERSION

1. Game Title

HIJIN-BATTLE

2. One-Sentence Game Idea

HIJIN-BATTLE is a short 3D samurai action game where the player fights through moonlit duels, learns to balance attack and defense, and survives against increasingly adaptive enemies.

3. Intended Player Experience

The intended experience is tense, readable, and skill-based. The player should feel like they are entering a night-time Japanese duel space where every close-range decision matters: attack too early and risk being punished, defend too much and lose guard resources, roll at the right moment to avoid danger, and watch enemy behaviour closely.

The mood is quiet and cinematic rather than arcade-like. The game uses dark scenery, gold UI accents, sword sounds, and moonlit backgrounds to support a samurai duel atmosphere.

4. Core Mechanic

The core mechanic is close-range sword combat built around timing. The player uses normal attacks, charged attacks, defense, guard value, roll evasion, and jump/repositioning while reading enemy AI behaviour.

5. What the Player Does Moment to Moment

• Moves around the duel space using WASD and the camera.

• Approaches or keeps distance from the enemy.

• Uses left mouse button for normal or charged attacks.

• Uses E to defend and manage the guard bar.

• Uses Space to roll away from danger or reposition.

• Uses right mouse button to jump.

• Watches enemy movement, attack wind-up, defense, evasion, and stagger feedback.

• Wins by defeating the enemy encounter, or fails by losing all HP.

6. Target Player

The target player is someone who enjoys short action games, samurai aesthetics, one-on-one combat, and learning enemy patterns. The vertical slice is suitable for players who want a compact but readable duel rather than a complex RPG.

7. Reference Games or Inspirations

Reference	Relevant Inspiration	How HIJIN-BATTLE Transforms It
Sekiro: Shadows Die Twice	Deflection, posture pressure, intense sword duels.	HIJIN-BATTLE uses a simpler guard bar and readable enemy pressure rather than copying the full posture system.
Ghost of Tsushima	Samurai atmosphere, cinematic presentation, strong visual theme.	The project takes inspiration from samurai mood and presentation but focuses on a compact Unity vertical slice.
Dark Souls / Soulslike combat	Spacing, commitment, punishment, recovery windows.	The project uses action locks and enemy attack recovery to make decisions feel committed.

Reference	Relevant Inspiration	How HIJIN-BATTLE Transforms It
Practice/training modes in action games	Safe place to test controls and enemy response.	The practice mode records damage, hits, defense, and evade data to support learning.

8. What Is Original or Creative About the Idea

The original contribution is the combination of a focused night-samurai duel theme, a practice mode that supports learning, and enemy AI that adapts within fixed probability ranges based on player behaviour. The project also treats the enemy model as a visual layer separated from gameplay control, allowing model replacement without rewriting AI.

Another creative element is the use of three difficulty readings as different playthrough tones: Easy as training, Normal as a formal duel, and Hard as a more aggressive test of timing.

9. Vertical Slice Plan

Area	Vertical Slice Goal
Start	Title scene and main menu with Start, Practice Mode, Settings, and Quit.
Practice	Training mode with non-lethal enemy and statistics panel.
Main Game	Playable battle scene with objective, health/guard UI, enemy target UI, pause, result flow.
Combat	Player attack, charged attack, jump, defend, roll, guard bar, damage, and death.
Enemy	AI with patrol, alert, chase, strafe, attack, defend, evade, stagger, return, and death.
Feedback	HUD, guard/HP bars, sword sound, guard sound, story/notification text.
Submission	GitHub-ready Unity project, README, contribution evidence, and playable build or build link.

10. Must-Have, Should-Have, Could-Have, and Cut-First Features

Priority	Features
Must-have	Playable title/menu flow; main battle scene; practice mode; player movement; attack; defend; roll; jump; enemy AI; win/fail/retry flow; HUD.
Should-have	Difficulty selection; records; settings; BGM; sword/guard SFX; custom font; story transitions; target information UI.
Could-have	More levels; more enemy types; more story art; more sound effects; camera shake; hit-stop; deeper boss phases.
Cut-first	Large open world, complex inventory, online features, advanced save slots, fully voiced dialogue, large multi-hour campaign.

11. Unity Development Plan

• Use existing Unity scenes and scripts rather than rebuilding the project architecture.

• Keep player combat in PlayerController and enemy behaviour in EnemyController.

• Use Resources for runtime-loaded audio, fonts, story images, and enemy visual prefabs.

• Use PlayerPrefs for lightweight settings, records, difficulty, and progression.

• Use runtime UI controllers for HUD, practice statistics, pause, result, and story panels.

• Use Git LFS for large Unity assets such as audio, images, models, and packages.

12. Main Systems/Scripts Expected to Build

System	Main Scripts
Player combat	PlayerController.cs, LockMoveState.cs
Enemy AI	EnemyController.cs, EnemyVisualController.cs, PlayerActionProfile.cs
Main-game flow	MainGameUIController.cs, MainGameEncounterController.cs, MainGameProgress.cs
Practice mode	PracticeStatsController.cs
Menu and scene flow	SceneFlowManager.cs, UI/MenuPanelController.cs, UI/TitlePageConreoller.cs
Settings	GameSettingsController.cs, UI/MenuPanelController.cs
Audio	MainGameBgmController.cs, MainGameSfxController.cs
Presentation	GameFontController.cs, simplecamera.cs, BullboardUI.cs

13. Asset/Resource Plan

Asset Area	Plan
Player	Use the existing player skin to keep the project consistent.
Enemies	Use supplied samurai/character model variants for normal, guard, elite, and boss-style enemies where animation compatibility allows.
Environment	Use the existing Japanese/night scene as the main duel space, with movement boundaries inside the building area.
UI	Use dark panels, gold rectangular borders, custom style font, and readable body text.
Audio	Use separate BGM for menu, practice, story, battle, and boss battle; use sword/guard SFX for combat feedback.
Story images	Use moonlit Japanese/samurai transition images as lightweight narrative support.
Fonts	Use the imported HIJIN style font selectively for title/menu/styled UI.

14. Legal, Ethical, Social, Accessibility, and Security Considerations

Area	Consideration
Legal	All third-party models, audio, fonts, and images must be credited with source and licence/permission. GitHub should not include assets that cannot be redistributed.
Ethical	AI assistance must be declared honestly. The final submission should show personal design decisions, testing, modification, and understanding.
Social	The game uses stylised sword combat and should be presented as fictional action rather than realistic violence.
Accessibility	Controls are listed in game; UI uses clear contrast; settings include volume and mouse sensitivity. Future work could add key rebinding and subtitles.
Security	The project avoids network features and does not collect user data. PlayerPrefs are used only for local settings/progress.

15. Development Schedule or Milestone Plan

Milestone	Goal
M1 - Project review	Understand existing scenes, scripts, controls, and prototype structure.
M2 - Practice mode	Add missing player abilities, practice UI, statistics, difficulty, and pause flow.
M3 - Main game	Create maingame scene, combat HUD, result flow, enemy target UI, and controls panel.
M4 - Enemy AI	Improve patrol, chase, attack, defend, evade, stagger, return, and difficulty profiles.
M5 - Progression and story	Add records, continue/new game, story images/text, and lightweight chapter flow.

Milestone	Goal
M6 - Audio/UI polish	Add BGM, SFX, font, settings, layout fixes, and environment presentation.
M7 - Testing and submission	Run playtests, fix visible bugs, prepare README, contribution evidence, build, and GitHub upload.